

# Complementi di Matematica e Calcolo Numerico

## A.A. 2024-2025

### Laboratorio 1 - Introduzione a MATLAB

**MATLAB = MAT(rix)-LAB(oratory)** è un ambiente integrato per il calcolo scientifico utilizzabile sia in maniera interattiva che come linguaggio di programmazione.

In Matlab ogni quantità (variabile) viene trattata come matrice.

In particolare una grandezza scalare (un numero reale) è una matrice  $1 \times 1$ .

Sono predefinite numerose funzioni di uso generale (**built-in functions**), e raccolte di funzioni dedicate ad uno specifico argomento (**toolboxes**).

Per informazioni su Matlab: [www.mathworks.com](http://www.mathworks.com)

Matlab è un software a pagamento ma l'Università degli Studi di Milano ha stipulato un contratto di licenza campus, che permette agli studenti di scaricarlo ed utilizzarlo sul proprio PC. Per informazioni su come fare si veda

[https://work.unimi.it/servizi/servizi\\_tec/79539.htm](https://work.unimi.it/servizi/servizi_tec/79539.htm)

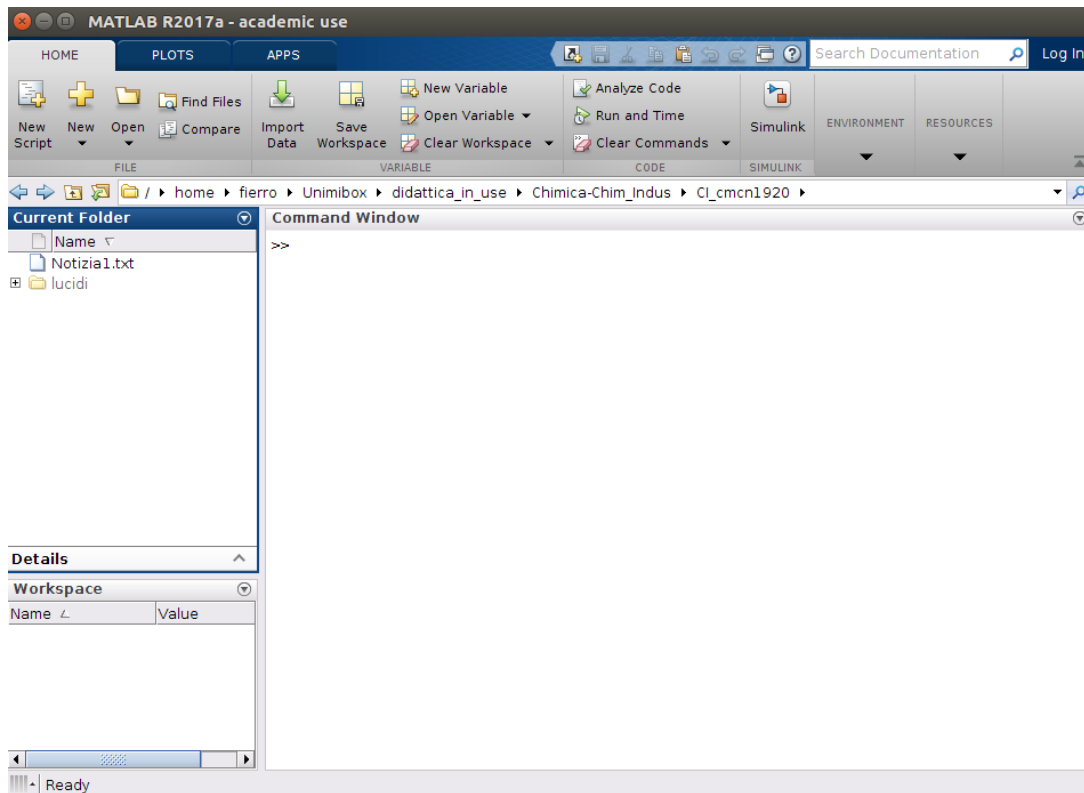
In alternativa esiste un software gratuito **Octave** che riproduce buona parte delle funzioni fondamentali di Matlab.

Per informazioni vedere [www.octave.org](http://www.octave.org).

## Matlab in modalità interattiva

All'avvio di Matlab si apre una finestra grafica normalmente divisa in diversi campi.

- In alto: barra dei diversi menù
- a sinistra-alto: current folder con la lista dei files contenuti nella directory corrente
- a sinistra-basso: workspace la lista delle variabili in memoria
- a destra: la finestra di lavoro o command window



La finestra di lavoro è caratterizzata dal prompt

>>

Tutto quanto inserito dopo il prompt verrà eseguito dopo aver premuto il tasto **enter**.

Se Matlab riconosce il comando digitato produrrà un *output* in caso contrario segnalerà un errore. In ogni caso il sistema ripropone al termine il prompt in attesa di un nuovo comando.

Il comando **quit** chiude Matlab.

I comandi Matlab **help** e **doc** permettono di ottenere informazioni dettagliate su qualsiasi comando o funzione di Matlab. Il comando **doc** mostra anche quali pacchetti (toolboxes) siano installati nella versione in uso.

Ad esempio:

```
>> help sqrt
```

```
sqrt - Square root
```

```
This MATLAB function returns the square root of each element  
of the array X.
```

```
Syntax
```

```
B = sqrt(X)
```

```
Input Arguments
```

```
X - Input array
```

```
scalar | vector | matrix | multidimensional array |  
table | timetable
```

Per cercare il nome esatto di un comando si usa `lookfor`. Ad esempio:

```
>> lookfor cosine
```

```
acos          - Inverse cosine, result in radians.  
acosd         - Inverse cosine, result in degrees.  
acosh         - Inverse hyperbolic cosine.  
cos           - Cosine of argument in radians.  
cosd          - Cosine of argument in degrees.  
.....
```

cerca i comandi nella cui descrizione appare la parola cosine  
(ATT.NE la documentazione di Matlab è in inglese!)

Il comando `diary` è molto utile in modalità interattiva in quanto permette di salvare la sessione di lavoro in un file. Digitando

```
>> diary laboratorio_1
```

tutto ciò che segue da quel comando in poi viene salvato in un file di testo che si chiama `laboratorio1` fintanto che non terminate la sessione con

```
>> diary off
```

## Scalari in Matlab

Matlab valuta espressioni e ne assegna il valore a variabili. Nel caso più semplice il valore è un numero reale.

Assegnazione di variabili:

```
>> z=6  
z =  
    6
```

z è il nome della variabile, 6 è il suo valore.

```
>> 6+2  
ans =  
    8
```

se non specificato il valore 8 dell'espressione viene assegnato alla variabile **ans** che contiene sempre l'ultimo valore non esplicitamente assegnato ad una variabile.

```
>> a=2.5;
```

Il ; alla fine dell'istruzione sopprime la visualizzazione a schermo del risultato (ma non l'esecuzione dell'operazione!). È sufficiente digitare il nome di una variabile e Matlab ne mostrerà il valore:

```
>> a  
a =  
2.5
```

## **I nomi delle variabili devono rispettare le regole seguenti:**

- contenere al massimo 31 caratteri
- non iniziare MAI con un numero
- non contenere spazi
- non contenere segni di punteggiatura ed operazione
- non contenere apostrofi, slash e backslash
- possono contenere l'underscore
- lettere maiuscole e minuscole sono caratteri differenti

**>> (who) whos**

(elenca le variabili attualmente attive in memoria) e dà alcune informazioni importanti sulle loro caratteristiche (tipo di oggetto, dimensioni ...)

**>> clear all**

cancella tutte le variabili attive in memoria.

**>> clear x**

cancella la variabile x

## Alcune variabili predefinite:

- `pi` (pigreco),  $\pi$
- `i, j` (unità immaginaria),  $i = \sqrt{-1}$

Ogni variabile (anche predefinita) può essere sovrascritta. Per ristabilire il valore predefinito è sufficiente utilizzare il comando `clear`.

```
>>pi  
3.1416
```

```
>> pi=5  
pi =  
    5  
>> clear pi  
>> pi  
3.1416
```

## Operazioni elementari

Sono definite le operazioni elementari:  $+$ ,  $-$ ,  $*$ ,  $/$ ,  $\wedge$  (elevamento a potenza).

```
>> a=3+2.5, b=5-3, d=3*4.2, e=3/2, f=2^3
```

Si osservi che le diverse assegnazioni di variabili inserite nella stessa riga di comando vanno separate con delle virgole.

Attenzione alle precedenze:

```
>> 3+2*4
```

```
ans=
```

```
    11
```

```
>> 3*2^4
```

```
ans=
```

```
    48
```

Per alterare l'ordine delle operazioni si utilizzano le parentesi tonde.

```
>> (3+2)*4
```

```
ans=
```

```
    20
```

```
>> (3*2)^4
```

```
ans=
```

```
   1296
```



Le espressioni possono dipendere da variabili già definite, ad esempio: se  $x = 8$  posso valutare l'espressione  $3x + 1$  ed assegnarla alla variabile  $y$ .  
In matlab:

```
>> x=8
x =
     8
>> y=3*x+1
y =
     2
```

### Attenzione:

Usualmente si scrive  $3x$  intendendo  $3 \cdot x$ , è importante non dimenticare l'operatore di moltiplicazione quando si inserisce il comando per valutare l'espressione in matlab, infatti in caso contrario si ottiene l'errore:

```
>> x=8;
>> 3x
3x
|
```

Error: Unexpected MATLAB expression.

## Numeri Complessi:

In Matlab sono definiti i numeri complessi, ovvero del tipo:

$$z = 5 + 3i$$

dove 5 è la parte reale di  $z$  e 3 la parte immaginaria ( $\text{Re}(z) = 5, \text{Im}(z) = 3$ ). Se la variabile predefinita **i**, contenente l'unità immaginaria, non è stata ridefinita, un numero complesso può essere scritto in Matlab nei modi seguenti:

```
>> z=5+3i
z =
    5.0000 + 3.0000i
```

```
>> y=2.5-2*i
y =
    2.5000 - 2.0000i
```

Pertanto l'operatore  $*$  in questo caso eccezionalmente può essere omissa.

Esistono funzioni predefinite di Matlab che operano sui numeri complessi, ad esempio se  $z = 5 + 3i$  abbiamo:

```
>> real(z)   restituisce 5
>> imag(z)   restituisce 3
>> conj(z)   restituisce il complesso coniugato: 5-3i
```

## Funzioni matematiche predefinite:

|                           |                                                                                        |
|---------------------------|----------------------------------------------------------------------------------------|
| <code>abs(x)</code>       | $ x $                                                                                  |
| <code>sqrt(x)</code>      | $\sqrt{x}$                                                                             |
| <code>nthroot(x,n)</code> | $\sqrt[n]{x} \in \mathbb{R} \quad \text{con } x \in \mathbb{R} \quad n \in \mathbb{R}$ |
| <code>exp(x)</code>       | $e^x \quad \text{con } e = 2.7182818284... \text{ costante di Nepero}$                 |
| <code>log(x)</code>       | $\ln(x)$                                                                               |
| <code>sin(x)</code>       | $\text{sen}(x)$                                                                        |
| <code>cos(x)</code>       | $\cos(x)$                                                                              |
| <code>tan(x)</code>       | $\tan(x)$                                                                              |
| <code>asin(x)</code>      | $\arcsen(x)$                                                                           |
| <code>...</code>          |                                                                                        |

Per visualizzare l'elenco completo:

```
>> help elfun
```

**IMPORTANTE:** Per calcolare la costante di Nepero  $e$

```
>> exp(1)
ans =
    2.7183
```

### Alcune osservazioni sull' uso delle funzioni:

**Oss.1:** `nthroot(x,n)` restituisce la radice  $n$ -esima reale di un numero reale  $x$ . Se  $x$  è negativo  $n$  deve essere un numero intero dispari.

Si osservi che

```
>> nthroot(-8, 3)
```

```
ans =
```

```
    -2
```

```
>> (-8)^(1/3)
```

```
ans =
```

```
    1.0000 + 1.7321i
```

nel primo caso si ottiene la radice reale  $(-2)$ , nel secondo una radice complessa di  $-8$ .

**Oss.2:** Se  $z=a+bi$  è un numero complesso per definizione si ha

$$|z| = \sqrt{a^2 + b^2}$$

```
>> z=2+3i
```

```
>> abs(z)
```

```
ans =
```

```
    3.6056
```

**Oss.3:** Se  $z$  è un numero negativo o complesso `log(z)` non dà errore ma restituisce il logaritmo complesso. Ad esempio

```
>> log(-1)
```

```
ans =
```

```
    0 + 3.1416i
```

## Esercizio 1

- Posto  $a = 3, b = 2$ , calcolare

$$\frac{3}{a+b}, \quad \frac{a+b}{2}, \quad \frac{a+b}{2a}, \quad \frac{1}{8^{\frac{1}{3}}}, \quad \sqrt[4]{64}$$

- Se  $x = 10, y = 5, z = 2$ , calcolare

$$\frac{3x - 2y}{5z^2} \quad (= 1)$$

- Se  $a = \frac{1}{3}, b = \frac{1}{5}$ , calcolare

$$\frac{a^{-3}}{(1 - b + 3a)^2} \quad (= 8.\bar{3})$$

- Assegnare alla variabile  $z$  il numero complesso  $7 - 5i$  e alla variabile  $w$  il suo complesso coniugato.

## Esercizio 2

Calcolare le seguenti variabili reali:

- $y = 2 \sin(x) \cos(x) - \cos(2x)$  con  $x = \pi/2$ ,  $(R : y = 1)$

- $y = \frac{x}{\sqrt[5]{x-9}}$  con  $x = 5$   $(R : y = -3.7893)$

- $y = \frac{e^{\sin(x^2)} + \cos(x)}{2\sqrt{x} + 5\ln(x)}$  con  $x = 10$ ,  $(R : y = -0.0133)$

**Esercizio 3** Supponendo di non disporre della variabile predefinita `pi` determinare il valore di  $\pi$  sfruttando le funzioni matlab `atan` e `log` e ricordando le relazioni

$$\arctan(1) = \frac{\pi}{4}$$

$$\log(-1) = \pi i \quad \rightarrow \quad |\log(-1)| = \pi$$

## Matlab in modalità interattiva

Vediamo le funzionalità di alcuni tasti, comandi o operatori che possono essere usati per lavorare in modo più rapido quando usiamo Matlab in modalità interattiva.

- se abbiamo commesso un errore nella digitazione di un comando è possibile correggerlo spostandosi lungo la linea su cui stiamo scrivendo utilizzando i tasti  $\leftarrow$ ,  $\rightarrow$ , cancellando quindi l'errore e riscrivendo la forma corretta.
- è possibile richiamare, senza doverli riscrivere di nuovo, i comandi digitati in precedenza utilizzando tasti  $\uparrow$ ,  $\downarrow$  (particolarmente utile se si utilizza più volte un comando lungo)
- è possibile iniziare a scrivere le prime lettere di un comando e completarlo con uno precedentemente digitato semplicemente premendo il tasto  $\uparrow$
- è possibile ripulire la finestra di comando da quanto scritto in precedenza digitando

```
>> clc .
```

Le variabili in memoria generate dai comandi digitati in precedenza resteranno invariate, questo comando funziona semplicemente come un 'cancellino per la lavagna'.

## Vettori in Matlab

Per assegnare una variabile di tipo vettore occorre associare al nome della variabile una lista delle sue componenti.

Per assegnare un **vettore riga** le componenti vanno separate da uno spazio (o da una virgola):

```
>> w=[1 2 3]
```

```
w =
```

```
1      2      3
```

oppure

```
>> w=[1, 2, 3];
```

per assegnare un **vettore colonna** le componenti vanno separate da un ; Possiamo interpretare il ; come il carattere che segnala la fine di una riga e rimanda a capo.

```
>> v=[1; 2; 3]
```

```
v =
```

```
1
```

```
2
```

```
3
```

Esistono altri modi di generare vettori riga. Per generare ad esempio vettori le cui componenti stanno alla stessa distanza l'una dall'altra abbiamo sostanzialmente due possibilità.

1) Per generare un vettore di numeri interi successivi possiamo scrivere:



```
>> v=[1:8]
```

```
v =
```

```
    1    2    3    4    5    6    7    8
```

Più genericamente per generare un vettore le cui componenti partono da un valore iniziale, si spostano con passo fissato fino al raggiungimento di un valore finale possiamo usare la seguente sintassi:

```
v=[valore_iniz:passo:valore_finale].
```

Ad esempio:

```
>> v=[1:0.5:3]
```

```
v =
```

```
    1.0000    1.5000    2.0000    2.5000    3.0000
```

Il passo può essere anche negativo:

```
>> >> v=[3:-0.5:1]
```

```
v =
```

```
    3.0000    2.5000    2.0000    1.5000    1.0000
```

Si osservi che il valore finale può non essere un elemento del vettore, dipende dal punto iniziale e dal passo, infatti il punto finale funge da barriera e non può essere superato

```
>> v=[1:0.7:5]
```

```
v =
```

```
    1.0000    1.7000    2.4000    3.1000    3.8000    4.5000
```

2) Il comando

`linspace(valore_iniziale, valore_finale, N)`

genera  $N$  valori equispaziati fra `valore_iniziale` e `valore_finale`. In questo caso i valori iniziale e finale sono sempre agli estremi del vettore generato e il 'passo' o la distanza tra due coefficienti consecutivi è scelto in modo da generare un totale di  $N$  coefficienti. Ad esempio

```
>> v=linspace(0,1,5)
      0      0.2500      0.5000      0.7500      1.0000
```

Ricordando che  $N$  punti equispaziati suddividono un intervallo  $[a, b]$  in  $N - 1$  sottointervalli di uguale ampiezza  $h = \frac{b-a}{N-1}$ , per generare l'ultimo vettore si può anche scrivere

```
h=1/4;
>> v=[0:h:1]
v =
      0      0.2500      0.5000      0.7500      1.0000
```

Il comando `zeros(n,1)` (`zeros(1,n)`) produce un vettore colonna (riga) di lunghezza  $n$  con elementi tutti nulli.

Il comando `ones(n,1)` (`ones(1,n)`) genera un vettore colonna (riga) con tutte le componenti pari a 1.

Ad esempio:

```
>> u=zeros(1,5)
u =
    0    0    0    0    0
```

```
>> w=ones(3,1)
w =
    1
    1
    1
```

Per conoscere la lunghezza di un vettore  $\mathbf{v}$ :

```
>> length(v)
ans =
5
```

Per controllare la dimensione di una variabile  $\mathbf{v}$ :

```
>> size(v)
ans =
1     5
```

il comando `size(v)` restituisce il numero di righe e di colonne di  $\mathbf{v}$ .

Ricordiamo che per Matlab ogni variabile è una matrice:

- una variabile scalare è una matrice  $1 \times 1$ ,
- un vettore colonna (riga) di  $n$  componenti è una matrice  $n \times 1$  ( $1 \times n$ ).

`length(v)` restituisce il valore più grande tra le due dimensioni di `v` e quindi per un vettore di  $n$  componenti il massimo tra 1 e  $n$ .

### Accedere alle componenti di un vettore

Per accedere ad una singola componente di un vettore scriviamo accanto al nome del vettore tra parentesi tonde l'indice che individua la componente che ci interessa. Ad esempio per la terza componente del vettore  $v$ :

```
>> v(3)
ans =
    0.5
```

**Attenzione:** in Matlab l'indicizzazione inizia da 1 e non da 0!

Matlab produce un messaggio di errore quando si cerchi di accedere ad una componente non definita, ad esempio:

```
>> z=v(0)
??? Subscript indices must either be real positive
integers or logicals.
```

**Nota:** esiste in Matlab la parola chiave `end` per accedere all'ultimo elemento di un vettore. Ad esempio, se  $v$  ha dieci elementi, `v(end)` equivale a `v(10)`.

Per accedere a più componenti dello stesso vettore contemporaneamente posso utilizzare un vettore di indici. Ad esempio

```
>> x=[-2, 4, 6, -3, 0, 1];
```

```
>> x(1:3)
```

```
ans =
```

```
-2      4      6
```

così estraggo le prime tre componenti di  $\mathbf{x}$ , infatti  $1:3$  è il vettore di indici  
[1 2 3]

```
>> x([4,2,1])
```

```
ans =
```

```
-3      4     -2
```

così estraggo la quarta, la seconda e la prima componente di  $\mathbf{x}$

```
>> x(1:2:5)
```

```
ans =
```

```
-2      6      0
```

così estraggo la prima, la terza e la quinta componente di  $\mathbf{x}$

## Alcune operazioni su vettori

Dati i vettori  $v=[1 \ 2 \ 3 \ 4]$  e  $w=\text{ones}(1,4)$ .

**Trasposizione** trasforma un vettore riga in colonna e viceversa

```
>> u=v'
```

```
u =
```

```
    1
```

```
    2
```

```
    3
```

```
    4
```

**Osservazione** Per mettere in colonna le componenti di un qualsiasi vettore si può utilizzare il comando `:`, ad esempio

```
>> u=[1 5 4 -8];
```

```
>> r=u(:)
```

```
r =
```

```
    1
```

```
    5
```

```
    4
```

```
   -8
```

se  $u$  fosse stato un vettore colonna il comando ne avrebbe generato semplicemente una copia in  $r$ .

**Somma (sottrazione)** algebrica tra vettori di ugual dimensioni

```
>> v+w
```

```
ans =
```

```
    2    3    4    5
```

```
>> v-w
```

```
ans =
```

```
    0    1    2    3
```

**Prodotto o divisione per uno scalare.**

```
>> 2*v
```

```
ans =
```

```
    2    4    6    8
```

```
>> v/2
```

```
ans =
```

```
 0.5000  1.0000  1.5000  2.0000
```

**Somma o sottrazione di uno scalare.**

```
>> v + 5
```

```
ans =
```

```
    6    7    8    9
```

## Esercizio 4

- generare gli interi da 28 a 80 con passo 1
- generare gli interi da 22 a -10 con passo -4
- generare 125 punti equispaziati tra -1 e 5
- assegnati i vettori  $u = [1, 2, 0, -3]$  e  $v = [3, 0, 2, 1]$  calcolare:  
la somma  $u + v$ , la differenza dei trasposti  $u' - v'$ ,  $5u$  e  $\frac{u+v}{10}$ .

## Esercizi di riepilogo:

1. Calcolare:

$$a = \frac{e^2 \cos(\pi/6) + 1}{4 + \ln(3)}, \quad (R = 1,4512)$$

$$\frac{a + \sqrt[3]{a}}{2a + 4} \text{ per } a = 8, \quad (R = 0.5)$$

$$y = e^{-2x} \cos(3x) - 3\sqrt{x+1} \text{ con } x = 3, \quad (R = -6.0023)$$

Parte reale, immaginaria e modulo della variabile complessa  
 $z = (-7) \wedge (1/5)$

2. • generare gli interi da -13 a 75 con passo 2  
• generare i punti tra -2.7 a 8.3 con passo 1.5 (cosa si osserva?)  
• generare 150 punti equispaziati tra -2 e 3
3. Determinare il vettore  $x$  dei punti che individuano 10 sottointervalli di uguale ampiezza dell'intervallo  $[0, 1]$ , in due modi differenti: utilizzando il comando `linspace` e la sintassi `[::]`